

Міністерство освіти і науки України

Відокремлений структурний підрозділ «Тернопільський фаховий коледж
Тернопільського національного технічного університету імені Івана Пулюя»

Циклова комісія комп'ютерних наук

ЗВІТ

про виконання лабораторної роботи №12

з дисципліни: “Організація баз даних та знань”

на тему: “Профілювання MongoDB”

Виконав:
студент групи КН-321
Ясній Ю. І.

Прийняв:
Сербін В. С.

Тернопіль 2025

Мета: набути навичок профілювання документів в MongoDB.

Хід роботи

Для виявлення повільних запитів та аналізу продуктивності бази даних було налаштовано профілювання. Встановлено рівень профілювання 1, який дозволяє фіксувати лише ті операції, час виконання яких перевищує встановлений поріг. У межах роботи цей поріг встановлено на рівні 120 мс.

Результат виконання команди показано на рисунку 12.1.

```
use yasni_yurii

db.setProfilingLevel(1, { slowms: 120 })
switched to db yasni_yurii
Enterprise yasni_yurii
```

Рисунок 12.1 – Встановлення рівня профілювання 1 із порогом 120 мс

Для генерації потенційних записів у журналі профілювання було виконано декілька запитів до колекції users, включаючи вибірку за віком та сортування за рейтингом. Результати виконаних запитів наведено на рисунку 12.2.

```
> MONDOSH
QueryShapematch: 'BCCA5F76C5E2667C654F41CF5DA33F2F2D9B46E98D0CE18046B304D692454A',
command: {
  find: 'users',
  filter: {
    rating: 4.5
  },
  '$db': 'yasni_yurii'
},
serverInfo: {
  host: 'DESKTOP-S56M6S',
  port: 27017,
  version: '8.2.2',
  gitVersion: '594f83ceec1f4385be9a690131412d67b249a8a'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalQueryMaxWriteLockingMemoryUsageBytes: 104857600,
  internalQueryMaxAwaitTimeSecs: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFrameworkControl: 'tryShuffleRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
Enterprise yasni_yurii
```

Рисунок 12.2 – Виконання довільних запитів до колекції users

Після виконання запитів здійснено перевірку спеціальної колекції system.profile, яка зберігає інформацію про повільні операції. Оскільки локальна база даних містить невелику кількість документів, запити виконувалися швидше за встановлений поріг у 120 мс, тому колекція не зафіксувала нових записів.

Результат запиту наведено на рисунку 12.3.

```
host: 'DESKTOP-S56M6S',
port: 27017,
version: '8.2.2',
gitVersion: '594f83ceec1f4385be9a690131412d67b249a8a'
},
serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
```

Рисунок 12.3 – Вивід даних профілювання з колекції system.profile

Для детального аналізу плану виконання запиту було застосовано метод `explain("executionStats")`. Проаналізовано запит пошуку користувачів за рейтингом. У результаті виявлено, що база даних використовує метод COLLSCAN (повне сканування колекції), що є неефективним для великих обсягів даних

Результат роботи `explain()` подано на рисунку 12.4.

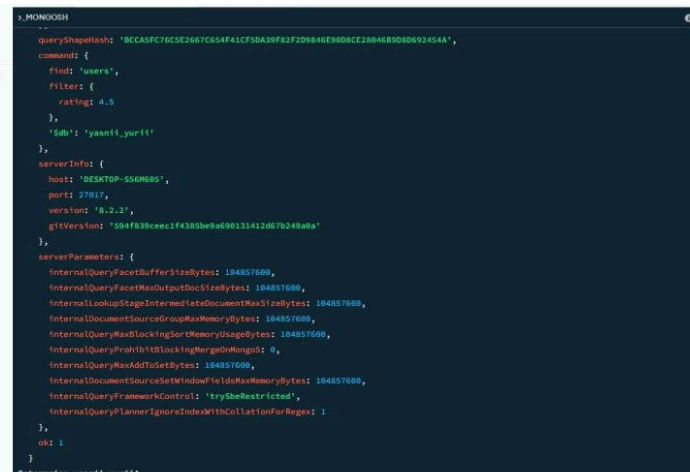


Рисунок 12.4 – Аналіз виконання запиту за допомогою explain()

Для покращення швидкодії запиту було створено індекс для поля rating. Після створення індексу виконано повторний виклик explain(). Отримані результати показали зміну стратегії виконання на IXSCAN (сканування індексу), що суттєво зменшило кількість перевірених документів та навантаження на систему. Результати аналізу подано на рисунку 12.5.

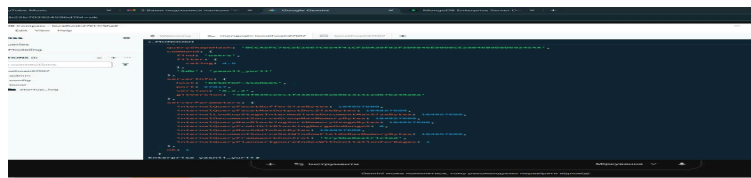


Рисунок 12.5 – Вплив створеного індексу на швидкодію запиту

Висновок: У ході лабораторної роботи було виконано налаштування профілювання MongoDB для моніторингу повільних запитів. Досліджено використання методу explain() для аналізу планів виконання запитів. Практично продемонстровано важливість індексації: створення індексу дозволило змінити метод сканування з повного перебору колекції (COLLSCAN) на ефективний пошук по індексу (IXSCAN), що оптимізує роботу бази даних.